

From Prompt to Workflow

VS Code + Cline + On-premise LLM으로
재구성한 일상 업무 자동화

Agentic Workflow

On-premise LLM

Markdown Rules

Human-in-the-loop

Git Workflow

발표일

2026. 04. 28

강의 시간

90분

슬라이드

37장

workflow.md

AGENTS.md

.clinerules/

```
1 # Agentic Workflow — System Overview
2
3 actor: Human
4   ↓ VS Code + Cline
5   ↓ On-premise LLM # Qwen3.5 계열
6   ↓ Tools: [ Files · Terminal · Git · Notes ]
7   → Human Approval Gate
8
9 # Rule Hierarchy
10 ~/.cline/rules/ # Global Rules
11 .clinerules/ # Project-local Rules
12 .cline/skills/ # On-demand Skills
13 memory-bank/ # Persistent Memory
14
15 # Daily Slash Commands
16 /email-triage → /daily-report → /git-wrapup
17
18 # "업무 구조가 AI를 움직인다"
```

| 우리가 매일 반복하는 질문과 업무의 한계

Section 1. Opening & Motivation — 10분

🕒 매일 반복되는 일상 업무 4가지



이메일 확인

수신함 분류, 우선순위 파악,
답장 초안 작성 반복



보고서 초안

일일 업무 정리, 진행 현황
요약, 다음 계획 작성



Git 커밋 메시지

변경 사항 정리, 커밋 메시지
작성, diff 요약 매번 반복



프로젝트 상태 정리

이슈 추적, 진행 현황 업데이트,
팀 공유 문서 작성



Context Switching 비용: 이메일 → 코드 → 보고서 → Git 사이의 잦은 전환으로 집중력과 시간이 낭비됩니다.

⚠️ 단순 AI 챗봇 사용의 한계



매번 같은 맥락을 처음부터 설명

"저는 연구원이고, 이 프로젝트는..."을 세션마다 반복해야 함



세션 종료 후 맥락 완전 소멸

어제의 작업 이력과 결정 사항이 다음 세션에서 기억되지 않음



일관성 없는 출력 형식

같은 요청도 매번 다른 포맷과 스타일로 응답해 후처리 비용 발생



업무 이력이 쌓이지 않는 일회성 도구

대화가 끝나면 배움이나 축적된 맥락이 남지 않아 재사용 불가

? **핵심 질문:** AI 챗봇에게 매번 같은 내용을 설명하고 있지 않은가? — 더 많은 프롬프트가 아니라, 더 나은 업무 구조 (Workflow Engineering)가 필요합니다.

I Agentic Workflow의 정의와 접근 방식

Section 1. Opening & Motivation

Workflow

- ✓ LLM과 도구가 **미리 정해진 경로**로 실행되는 시스템
- ✓ 결과와 절차의 **예측 가능성**이 매우 높음

Agent

- ✓ LLM이 스스로 절차와 도구를 **동적으로 결정**하는 시스템
- ✓ 높은 **자율성**을 가지며 복잡한 목표 해결에 적합

TODAY'S APPROACH

Workflow-heavy Agentic System

사람이 설계한 구조
(Rules / Skills / Workflows)

+

AI의 지능적 실행
(Tool Use / Logic Reasoning)



"완전 자율 AI"가 아니라, 사람이 설계한 구조 위에서 AI가 움직이는 시스템을 지향합니다.

왜 Daily Office Workflow에 에이전트가 필요한가

Section 1. Opening & Motivation — 반복 업무의 특성과 맥락 전환 비용 (Context Switching Cost)

에이전틱 하니스에 적합한 업무의 3가지 특성



반복성 高 (High Repetition)

이메일 확인, 보고서 초안, Git 커밋 메시지, 프로젝트 상태 정리 — 매일 같은 패턴이 반복된다. 구조화하면 재사용 가능한 Workflow가 된다.



문서로 추적 가능 (Document-trackable)

Git 히스토리, 작업 노트, 이메일 기록 — 모든 업무는 텍스트 기반으로 기록된다. AI가 참조하고, 사람이 검증할 수 있는 근거가 남는다.



사람 승인으로 종료 (Human-approved closure)

전송·push·배포 전 명시적 승인 단계를 설계할 수 있다. 완전 자율화(autonomy) 없이도 자동화의 효율을 누릴 수 있다.

맥락 전환 비용 (CONTEXT SWITCHING COST)

매일 반복되는 업무 전환 패턴 — 매번 AI에게 맥락을 재설정해야 하는 비용



이메일
확인



코드
작업



보고서
작성



Git
커밋

⚠ 전환할 때마다 AI에게 **같은 배경을 재설정**해야 하는 비용이 누적된다

✖ Without Harness

- 매번 맥락을 직접 설명
- 결과 형식이 매번 달라짐
- 업무 이력이 축적되지 않음

✔ With Harness

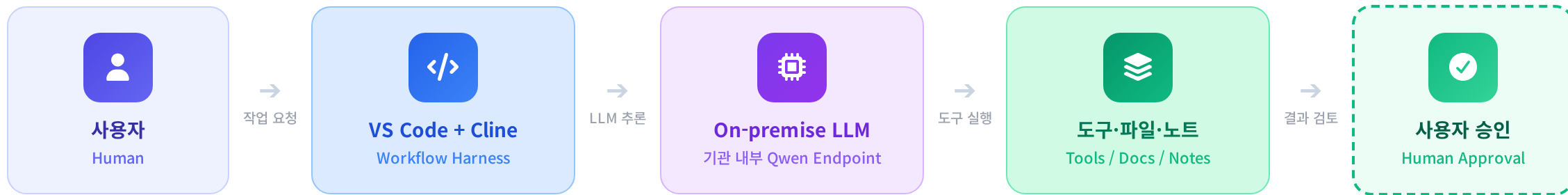
- Rules로 맥락이 자동 주입
- 일관된 출력 포맷 유지
- 작업 이력이 다음 입력으로

💡 **완전 자동화가 아닌 반자동화** — 반복성·추적 가능성·사람 승인이라는 세 조건을 갖춘 업무가 Agentic Workflow의 최적 출발점이다.

| 개인 작업 환경 아키텍처: VS Code와 Cline의 결합

System Architecture Overview — 요청부터 사용자 승인까지의 전체 흐름

ARCHITECTURE FLOW



🔌 Cline 도구 접근 권한 (Tool Access)



파일 읽기/쓰기
File R/W



터미널 명령 실행
Terminal Commands



브라우저 자동화
Browser Automation



MCP 서버 연결
MCP Server

🛡️ Zero-trust 원칙

- ✓ 모든 변경은 **사용자 명시적 승인** 후 실행
- ✓ 코드는 Cline 서버에 전송되지 않음 (Client-side)
- ✓ Provider 교체로 모델 lock-in 없이 운영 가능

Cline: 단순 코딩 도구를 넘어선 Workflow Harness

개인화된 업무 실행 환경으로서의 Cline과 4대 핵심 기능

Cline은 단순한 "채팅 보조 도구"가 아닙니다. VS Code 내에서 **파일, 터미널, 브라우저에 직접 접근**하여 실제 작업을 수행하고 결과를 검증하는 **AI 에이전트 실행 환경(Harness)**입니다.



파일 편집 및 관리

- 프로젝트 파일 읽기/쓰기
- 실시간 코드 수정 및 생성
- 문서 구조 분석 및 리팩토링



터미널 명령 실행

- Shell 명령 자동 실행
- Exit code 추적 및 에러 분석
- 빌드, 테스트, 배포 CLI 연동



브라우저 자동화

- 웹 검색 및 최신 정보 수집
- 웹 UI 상호작용 및 스크린샷
- MCP 서버를 통한 외부 툴 연결



Human-in-the-loop

- 모든 실행에 대한 사용자 승인
- Zero-trust 기반 클라이언트 구조
- 작업 이력 검토 및 롤백 지원

66

"Cline은 단순한 코딩 보조가 아니라 규정, 기술, 워크플로를 통합하는 **작업 하니스(Work Harness)**입니다."

AGENTIC WORKFLOW ENGINE

| 연구소 환경에서 On-premise LLM을 사용하는 이유

Section 2. System Overview: VS Code + Cline + On-premise LLM

Privacy (데이터 보안)

- ✓ 내부 연구 데이터 및 미공개 코드의 외부 유출 원천 차단
- ✓ 민감한 개인 정보 및 기관 기밀 사항의 내부 처리

Governance (거버넌스)

- ✓ IAM 및 네트워크 정책을 통한 정교한 접근 권한 제어
- ✓ 모델 사용 이력에 대한 완전한 감사(Audit) 로그 확보

Reproducibility (재현성)

- ✓ 공급사 정책과 무관한 특정 모델 버전의 고정 및 유지
- ✓ 동일한 파라미터와 설정으로 연구 결과의 일관성 보장

Compliance (컴플라이언스)

- ✓ 조직 고유의 보안 가이드라인 및 규제 준수 용이
- ✓ NIST AI RMF 기반의 Human Oversight 절차 설계

“ On-premise LLM은 단순히 '외부망 차단'을 넘어, 데이터 경계 설정, 모델 버전 제어, 감사 가능성을 확보하여 조직의 거버넌스 체계 내에서 AI를 운영하기 위한 필수적인 선택입니다.

Cloud LLM vs On-premise LLM 거버넌스 비교

Section 2. System Overview: VS Code + Cline + On-premise LLM

비교 항목	Cloud SaaS LLM (OpenAI, Claude 등)	On-premise LLM (기관 내부 구축)
 데이터 처리 위치	외부 공급사 서버 (Public Cloud)	기관 내부 인프라 (Air-gapped 가능)
 모델 버전 관리	공급사 업데이트 일정에 의존	자체 결정 및 특정 버전 고정 가능
 접근 권한 제어	API Key 기반 (제한적 제어)	IAM + 내부 네트워크 정책 (세밀한 제어)
 감사 로그 (Audit)	공급사 제공 범위 내에서만 확인	입출력 전문 및 시스템 로그 완전 제어
 모델 커스터마이징	제한적인 Fine-tuning 또는 RAG	Full Fine-tuning 및 가중치 직접 관리
 가용성 및 지연시간	인터넷 연결 및 외부 서비스 상태 의존	내부 네트워크 기반의 안정적 가용성

연구소 환경에서의 전략적 선택

온프레미스 LLM은 단순한 보안을 넘어 **데이터 경계 확립**, **모델 버전 관리의 재현성**, **조직별 Oversight 설계**를 위해 필수적입니다. 특히 미공개 연구 데이터와 내부 소스 코드 보호를 위한 NIST AI RMF 기반의 거버넌스 체계를 구축하기에 가장 적합한 환경입니다.

| Prompting에서 Workflow Engineering으로의 전환

Section 3. From Prompting to Workflow Engineering

단순한 대화형 요청을 넘어, AI가 지속적으로 참조할 수 있는 **운영 체계**를 구축합니다.

항목	💬 단순 Prompting	⚙️ Workflow Engineering
지속성	세션 내 1회용 (One-off)	재사용 가능한 문서 (Reusable)
맥락 전달	매번 직접 설명 (Manual)	규칙·메모리로 자동 주입 (Automatic)
결과 형식	임의적 (Arbitrary)	미리 정의된 포맷 (Defined)
업무 추적	기록 없음 (No tracking)	Task history + Notes 추적 (Trackable)
팀 공유	어려움 (Hard to share)	Markdown 문서로 즉시 공유 가능



핵심 철학의 전환

"프롬프트는 세션 안에서 던지는 요청이고, **Workflow Engineering**은 AI가 매번 참조할 수 있는 운영 체계를 구축하는 것입니다."

| Human-readable 및 AI-readable Markdown 문서의 힘

Section 3 · Workflow Engineering — 사람과 Cline이 동시에 읽고 따르는 운영 계약 문서

HUMAN READER 사람이 읽을 때

- ✓ 업무 절차와 규칙을 자연어로 이해하고 동료와 공유
- ✓ 프로젝트 맥락, 제약 조건, 승인 절차를 문서로 파악
- ✓ 팀 온보딩·협업 자료이자 업무 운영 매뉴얼로 활용

 README.md · 업무 매뉴얼 · 회의록



AI AGENT READER Cline이 읽을 때

- ⚡ 행동 지침·제약 조건을 컨텍스트에 자동으로 주입
- ⚡ 출력 포맷, 보안 규칙, 승인 절차를 일관되게 준수
- ⚡ 세션이 바뀌어도 동일한 업무 맥락과 규칙을 유지

 AGENTS.md · .clinerules/ · skills/



문서화 = 업무 구조화

AI에게 일을 맡길 수 있는 실질적 기반



운영 계약 (Working Agreement)

에이전트 행동 방식을 문서로 고정



Cline의 공식 인식 구조

AGENTS.md를 rule type으로 자동 인식



주석이 아닌 실행 지침

에이전트 행동을 직접 제어하는 규칙

| 업무 맥락을 에이전트가 이해할 수 있는 구조로 재설계

Section 3 — From Prompting to Workflow Engineering

🔄 재설계 3단계 프로세스

1

업무 습관 파악

반복되는 절차, 판단 기준, 제약 조건을
명확히 인식하고 언어화



2

Markdown으로 문서화

사람도, AI도 읽을 수 있는 구조화된
텍스트로 변환 · 저장



3

에이전트가 참조 · 실행

Cline이 문서를 읽고 일관된 방식으로
업무 절차 수행

💡 문서화는 AI를 위한 게 아니라 — 업무 자체를 더 명확하게 만드는 과정

🔗 업무 습관 → Cline 문서 유형 변환 예시



이메일 처리 습관 → Rules (항상 활성화)

"메일은 전송 전 반드시 검토 요청" → `.clinerules/` 에 명시



보고서 작성 방식 → Skills (온디맨드 로드)

"사실 근거 먼저 수집 후 문장화" → `.cline/skills/daily-report/` 에 명시



Git 커밋 전 확인 절차 → Workflows (단계별 실행)

"status → diff → risk check → commit" → `/git-daily-wrapup` 으로 호출



프로젝트 맥락 유지 → Memory Bank (세션 간 지식 유지)

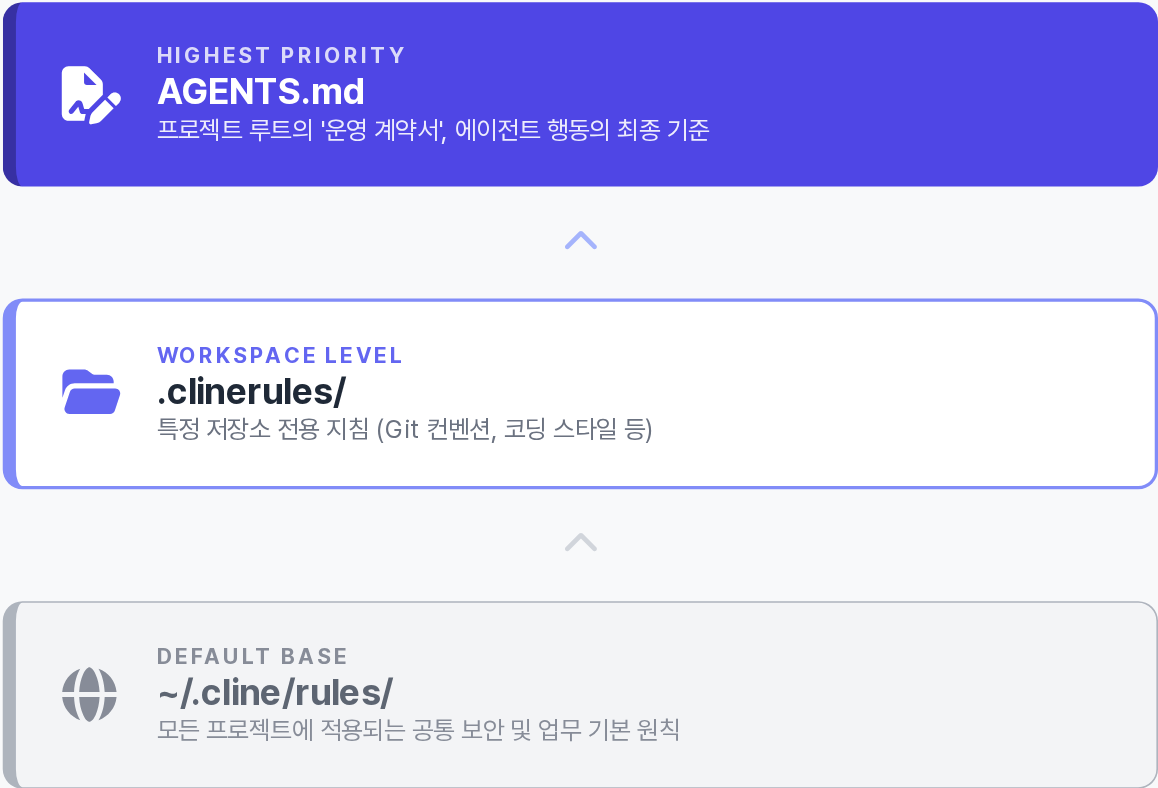
진행 상태, 의사결정 이력 → `memory-bank/progress.md` 에 누적 저장

“ 업무 절차를 문서화하면 사람 동료도, AI 에이전트도 따를 수 있다 — "AI를 잘 쓰는 것"보다 "내 업무 맥락과 절차를 잘 구조화하는 것"이 핵심

Cline Rule 파일의 계층 구조와 우선순위

Section 4. Global and Local Markdown Rules for Cline

우선순위 (Priority Overriding)



| Global Rules 설정 예시: 공통 업무 원칙 정의

Section 4. Global and Local Markdown Rules for Cline



기본 언어 및 출력 구조

모든 상호작용의 기본 언어를 한국어로 고정하고, 보고서나 요약 시 일관된 Markdown 구조(요약/액션/단계)를 유지하도록 강제합니다.



보안 및 개인정보 보호

민감 데이터 발견 시 익명화를 의무화하고, 외부 API 호출이나 전송 시 반드시 사용자의 명시적 승인을 받도록 설계합니다.



운영 표준 (Standard Ops)

Git 커밋, 메일 전송 등 위험도가 있는 작업은 단계별 진행 후 최종 확인 단계를 거치는 'Safety Gate'를 구축합니다.

~/cline/rules/AGENTS.md

AGENTS.md

Working agreements

- 기본 응답 언어는 한국어로 한다.
- 메일 전송, Git push, 배포는 반드시 명시적 승인 뒤에만 제안한다.
- 결과물은 가능한 한 요약, 액션 아이템, 다음 단계 구조로 정리한다.
- 민감 정보가 있으면 익명화된 샘플로 대체한다.

Interaction Style

- 단계별 진행, 각 단계마다 확인 요청
- 불확실한 경우 추정 대신 질문

Project-specific Rules: 특정 업무 맞춤형 지침

Section 4 · Global & Local Markdown Rules for Cline — `.clinerules/` 디렉토리 기반 프로젝트별 독립 운영

Git Convention

Conventional Commits

- › Commit 접두사: `feat / fix / docs / refactor / test`
- › Branch: `feature/[issue-number]-[short-desc]`
- › Merge Request: 코드 리뷰 **1인 이상** 승인 필수

Coding Style

Python · Fortran

- › Python: PEP 8 준수, `type hints` 필수
- › Fortran: 모듈 단위 분리, `implicit none` 명시
- › 함수 Docstring: **Google style** 통일

Report Format

연구노트 · 보고서

- › 섹션 구조: **목적 → 방법 → 결과 → 다음 단계**
- › 수치 데이터: 단위 명시 필수
- › 연구노트 형식: 프로젝트별 독립 적용



`.clinerules/` (Local) 규칙은 Global Rules보다 우선 적용 — 동일한 Cline이 서로 다른 프로젝트에서 다르게 동작

`.clinerules/10-research-project-policy.md`

Markdown

```
# .clinerules/10-research-project-policy.md
# 적용 범위: [연구 프로젝트명] workspace only
# Global Rules보다 우선 적용됨
```

Coding Convention

- Python: PEP 8 준수, `type hints` 필수
- Fortran: 모듈 단위 분리, `implicit none` 명시
- Docstring: **Google style** 통일

Git Workflow

- Commit message: `feat/fix/docs/refactor/test` 접두사 사용
- Branch naming: `feature/[issue-number]-[short-desc]`
- Merge request: 코드 리뷰 **1인 이상** 필수

Report Format

- 섹션 구조: **목적 → 방법 → 결과 → 다음 단계**
- 수치 데이터: 단위 명시 필수

Security Policy

- 미공개 연구 데이터: 익명화 샘플로만 처리
- 외부 API 호출: 명시적 승인 후에만 허용

 **같은 Cline, 다른 행동** — 연구 프로젝트에서는 Fortran/Python 규칙을, DevOps 레포에서는 Docker/CI 규칙을 각각 로컬 파일로 정의하여 프로젝트 맥락에 맞는 일관된 출력을 보장합니다.

I 데모 시나리오 개요 및 안전한 실행 원칙

Section 5. Demonstration Scenarios

Demo Safety Protocol: "Zero-Trust Execution"

Sanitized Data

Explicit Approval

No Live Secrets

오늘의 데모는 실제 운영 환경과 동일한 **에이전트 제약 조건** 하에 진행됩니다. 단순히 결과의 품질을 보는 것이 아니라, 사용자가 설정한 **Markdown Rules**가 어떻게 **AI의 행동을 통제하고 위험을 방지하는지** 확인하는 것이 핵심입니다.



Demo 1. 일일 체크업

Email Summarization

- ✓ 이메일 샘플 데이터 분류
- ✓ Action Item 자동 추출
- ✓ Daily Note로 지식 전환



Demo 2. 일일 보고서

Report Preparation

- ✓ 작업 이력(Git/Note) 수집
- ✓ Memory Bank 기반 초안
- ✓ Markdown 포매팅 최적화



Demo 3. Git 워크플로

DevOps Automation

- ✓ Repo 상태 및 변경 요약
- ✓ Risk Check & Commit
- ✓ Issue Tracker 연동 개념

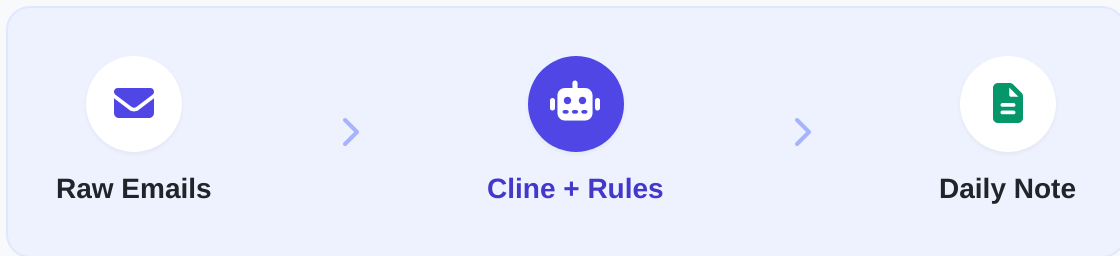
 모든 데모는 **Human-in-the-loop** 원칙을 준수합니다. Cline은 제안만 수행하며, 실제 파일 수정, 메일 전송, 커밋은 사용자의 명시적 승인 버튼 클릭 시에만 실행됩니다. 또한, 민감한 내부 정보는 **generic placeholder**로 대체된 안전한 샌드박스 환경에서 시연됩니다.

I Demo 1: 이메일 요약 및 일일 체크업 자동화

Section 5. Demonstration Scenarios

🔧 에이전트 동작 원리

- ✅ 수신된 메일을 긴급도(P0~P3)와 주제별로 자동 분류
- ✅ 핵심 내용 요약 및 즉각적인 **Action Item** 추출
- 💚 **Global Rules** 적용: 분류 후 사용자 승인 대기 (자동 전송 금지)
- 🏠✅ 분류 결과를 마크다운 기반 **Daily Note**로 자동 변환



OUTPUT: DAILY-CHECKUP.MD

Markdown Preview

Daily Checkup: 2026-04-27

P0: Urgent Action Required

Subject: [Critical] Internal Server Migration Schedule

- 내일 오전 10시 서버 점검 예정.
- **Action:** 연구 데이터 백업 확인 및 팀 공유 필요.

P1: Follow-up Needed

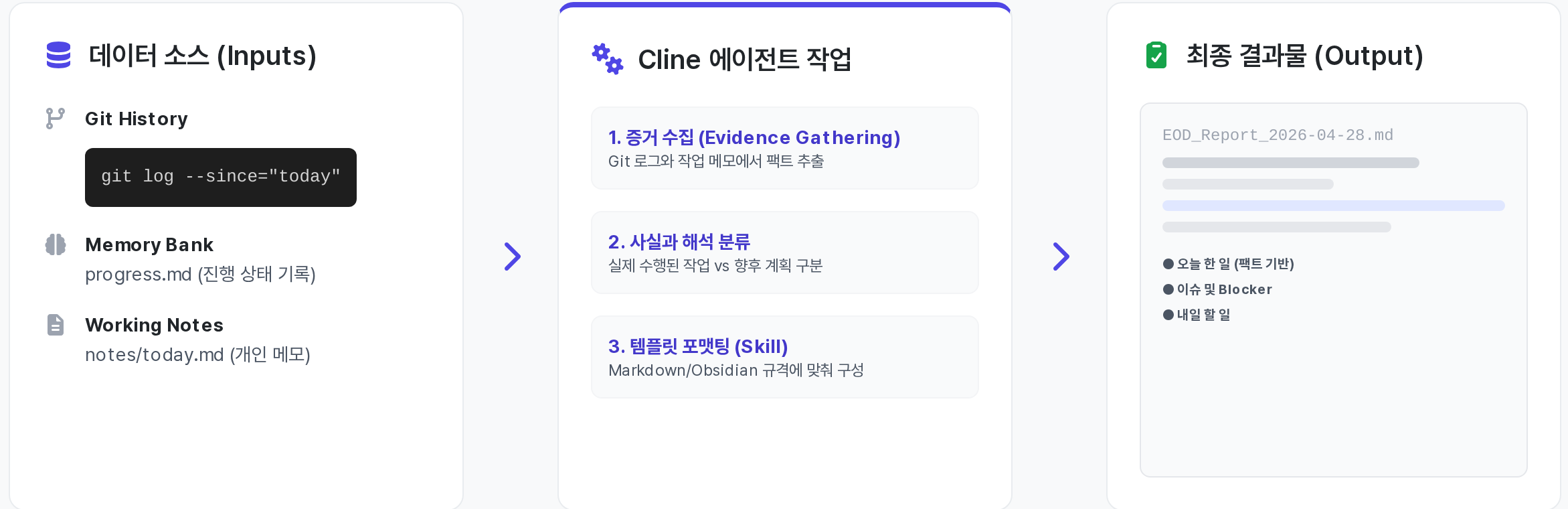
Subject: Research Collaboration Inquiry (Univ. A)

- 공동 연구 협약서 초안 검토 요청.
- **Action:** 법무팀 가이드라인 확인 후 회신 준비.

 Cline Note: 5개의 메일을 요약했습니다. 회신이 필요한 항목은 2건입니다.

I Demo 2: 작업 이력을 활용한 일일 보고서 초안 생성

Section 5. Demonstration Scenarios



💡 **핵심 가치:** 보고서가 갑자기 생긴 것이 아니라, **작업 증거 수집(Evidence) → 포매팅 Skill**의 결합으로 만들어진 시스템적 결과물입니다.

I 에이전틱 워크플로 구현을 위한 디렉토리 구조

규칙, 스킬, 워크플로, 메모리 बैं크의 실제 프로젝트 내 배치

PROJECT STRUCTURE

- ✓  office-agent-demo/
 -  AGENTS.md
 - ✓  .clinerules/
 -  10-office-policy.md
 -  workflows/
 - >  .cline/skills/
 - ✓  memory-bank/
 -  projectbrief.md
 -  progress.md
 -  inbox-demo/
 -  notes/
 -  repo-demo/

COMPONENT DEFINITIONS



AGENTS.md: 운영 계약서

프로젝트 루트에서 에이전트의 기본 행동 원칙, 언어 설정, 보안 수칙을 정의합니다. 사람과 AI가 공유하는 가장 상위 수준의 업무 지침입니다.



.clinerules/ & workflows/: 정책 및 절차

프로젝트 특화 규칙(Coding Style, Git Convention)과 슬래시 커맨드(/daily-report)로 호출 가능한 단계별 실행 절차를 저장합니다.



.cline/skills/: 온디맨드 기술

특정 상황에서만 로드되는 전문 지침입니다. 모든 규칙을 항상 로드하여 컨텍스트를 낭비하는 대신, 필요할 때만 기능을 확장합니다.



memory-bank/: 장기 기억 장치

프로젝트의 맥락(Context), 진행 상태(Progress), 주요 패턴을 기록하여 세션이 끊겨도 에이전트가 업무의 연속성을 유지하도록 돕습니다.

Memory Bank와 작업 이력을 통한 장기 기억 유지

Section 6 · Interactive Feedback Loop — 세션을 넘어 지식과 맥락을 지속적으로 축적하는 구조

MEMORY BANK 구성 파일 (5종)



projectbrief.md

프로젝트 목적 · 범위 · 핵심 요구사항 — 에이전트의 기본 방향 정의

기본 문서



activeContext.md

현재 작업 포커스 · 진행 중인 결정사항 — 가장 자주 갱신되는 파일

실시간 갱신



progress.md

완료된 작업 · 남은 작업 · 알려진 이슈 — 업무 진행 상황 추적

진행 추적



systemPatterns.md

아키텍처 패턴 · 설계 결정 이력 — 일관된 기술 선택 유지

패턴 기록



techContext.md

기술 스택 · 환경 설정 · 의존성 정보 — 실행 환경 맥락 보존

환경 맥락



Tasks

세션 단위 작업 기록

- 대화 흐름 및 의사결정 과정 전체 기록
- 코드 변경 · 터미널 명령 실행 이력 보존
- 다음 세션에서 이전 맥락을 참조 가능



Checkpoints

Shadow Git 파일 상태 스냅샷

- 실제 Git 히스토리 오염 없이 파일 상태 저장
- 에이전트 오류 발생 시 즉시 이전 상태로 복원
- 대화 맥락 유지 + 코드만 선택적 롤백 가능

“ 대화 맥락은 유지하면서 코드만 되돌릴 수 있다

Memory Bank + Checkpoints의 핵심 가치 — Cline의 가장 강력한 장기 기억 메커니즘

Human-in-the-loop: 에이전트와 사람의 경계 설계

Section 6 · Interactive Feedback Loop — 자동화(Automation) ≠ 자율성(Autonomy)



AUTOMATION ZONE

에이전트에게 맡길 일

자동 실행 가능



데이터 수집 및 문서 분류

이메일 요약, 파일 읽기, 우선순위 분류, 정보 정리



초안 작성 및 형식 변환

보고서 초안, 커밋 메시지 후보 3개 제시, 포맷 정리



상태 확인 및 로그 분석

`git status / log / diff` 실행, 진행 상태 요약



읽기 전용 명령 실행

무해한 로컬 테스트, 조회 명령, 메모리 뱅크 업데이트



승인
게이트



HUMAN REVIEW REQUIRED

사람이 검토해야 할 일

명시적 승인 필수



전송 / 배포 / 삭제

이메일 발송, git push, 서버 배포, 파일 삭제 — 반드시 확인



최종 판단 및 의사결정

보고서 최종 승인, 방향 설정, 우선순위 결정, 책임 귀속



민감 정보 처리

개인정보, 미공개 연구데이터, 내부 보안 정보 취급



출력 실패 시 정제 및 재실행

체크포인트 복원 → 지침 수정 → 재실행 판단



핵심 원칙 — 자동화(Automation) ≠ 자율성(Autonomy)

✓ 승인 지점 사전 설계

👤 사람이 최종 책임자

📋 책임 경계 명문화

| 거버넌스 및 리스크 관리의 핵심 원칙

Section 7. Governance, Boundaries, and Risk Management



Core Philosophy

"Trust the Workflow, not Blind Autonomy"

맹목적인 자율성보다 검증된 업무 절차(Workflow)를 신뢰하는 설계가 거버넌스의 시작입니다.



Human Oversight

NIST AI RMF 기반: AI 위험 관리 프레임워크에 따른 관리 절차 준수

승인 게이트: 전송, 배포, 삭제 등 중요 작업은 반드시 명시적 승인 필요

책임 소재: 결과물에 대한 최종 판단 및 책임은 사람이 담당함을 명시



AI System Inventory

자산 기록: 사용 모델 종류, 버전, Access Mode(vLLM 등) 기록

데이터 경계: 민감 데이터 처리 범위 및 익명화 샘플링 원칙 수립

접근 제어: IAM 및 네트워크 정책을 통한 온프레미스 엔드포인트 보호



Documentation as Control

Markdown Rules: 거버넌스 원칙을 AI가 읽을 수 있는 문서로 정의

Audit Log: Cline의 Task history와 Checkpoint를 통한 사후 감사

규정화: 단순 편의 기능이 아닌 조직의 Oversight를 문서로 고정

핵심 원칙: 온프레미스 환경에서도 데이터의 물리적 위치보다 '누가, 어떤 규칙으로, 어떻게 검증하는가'가 거버넌스의 본질입니다.

| Cline Rules를 활용한 보안 및 권한 통제

Section 7. 거버넌스 및 리스크 관리

명시적 금지 행동

- ❌ 메일 전송 및 외부 API 호출 금지 (자동 실행 차단)
- ❌ Git Push 및 서버 배포 전 반드시 명시적 승인 요청
- ❌ 로그 출력 시 민감 정보(Key, Credential) 익명화

권한 및 신뢰 정책

- ✔ **Publisher Trust:** 신뢰할 수 있는 개발사 확장만 허용
- ✔ **Allowed Extensions:** 조직 내 승인된 도구 리스트 관리
- ✔ **Workspace Trust:** 비인가 코드 실행 방지를 위한 신뢰 설정

데이터 경계 설정

- ✔ **.clineignore 활용:** 환경 변수(.env), DB 덤프 파일 제외
- ✔ **Context Control:** 불필요한 파일 접근 차단으로 보안 강화
- ✔ **Data Sanitization:** 에이전트 전송 데이터의 사전 필터링

 에이전틱 워크플로의 완성은 "무엇을 할 수 있는가"가 아니라, "무엇을 해서는 안 되는가"를 정의하는 것에서 시작됩니다.

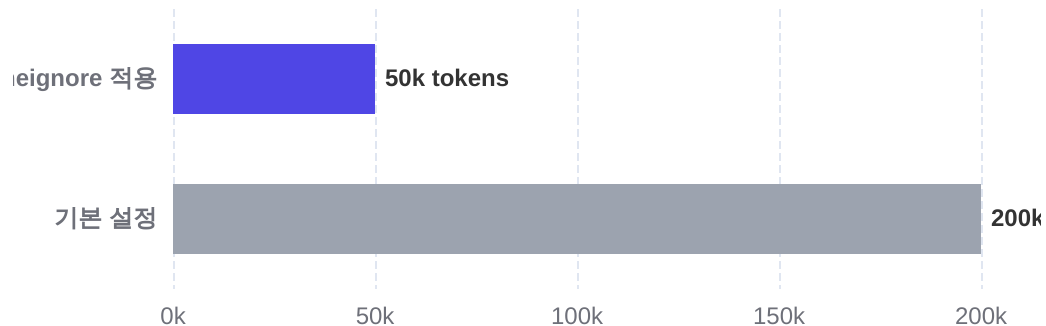
컨텍스트 관리와 실무 적용 시의 한계점

Context Optimization & Technical Constraints

🔧 컨텍스트 최적화 전략

- **.clineignore 활용:** 불필요한 파일(node_modules, build 등)을 제외하여 초기 로드 컨텍스트를 대폭 축소
- **명확한 Task Boundary:** 하나의 세션에서 처리할 작업 범위를 좁게 설정하여 토큰 소비 관리
- **가벼운 규칙 유지:** Rules가 무거워지면 매 요청마다 컨텍스트가 낭비되므로 핵심 지침 위주로 구성

.clineignore 적용 전후 컨텍스트 크기 변화 (Tokens)



⚠️ 실무 적용 시의 한계점

1. 성능 및 비용 이슈 (Latency & Cost)

컨텍스트가 임계치를 넘으면 모델의 응답 속도가 현저히 느려지고, 온프레미스 GPU 자원 소모가 급증하며 디버깅 난이도가 상승합니다.

2. 실험적 기능의 불안정성 (Experimental Skills)

Skills 기능은 현재 실험적 단계로, 업데이트에 따라 동작 방식이 변경될 수 있으며 트리거 조건이 일관되지 않을 수 있습니다.

3. 인프라별 기능 제약 (Provider Gaps)

내부 Qwen 엔드포인트 환경에서는 Cline의 일부 Web Tools나 외부 API 연동 기능이 네트워크 보안 정책상 제한될 수 있습니다.



핵심 극복 원칙

"짧고 살아 있는 규칙(Living Rules), 선명한 작업 경계(Task Boundary), 가벼운 컨텍스트 풋프린트(Light Footprint)를 유지하십시오."

| 결론: AI 에이전트를 위한 업무 운영 체계 구축

From Prompt to Workflow: 개인화 Agentic Workflow의 핵심 요약



모델보다 중요한 업무 맥락의 구조화

에이전틱 워크플로의 핵심은 모델 성능이 아니라, 사람이 남기는 **업무 맥락**을 AI가 읽을 수 있도록 **문서화·구조화**하는 방식에 있습니다.



Cline: 단순 도구가 아닌 업무 하니스(Harness)

Cline은 단순한 채팅 도구가 아닙니다. **규칙(Rules)**, **기술(Skills)**, **메모리(Memory)**를 묶어 실제 업무를 실행하는 워크플로 엔진입니다.



On-premise LLM: 거버넌스와 재현성

온프레미스 환경은 보안을 넘어 **데이터 경계 보호**, **모델 버전 관리**, **조직 내 업무 재현성**을 담보하는 전략적 운영 선택입니다.

NEXT STEPS: 다음 주에 바로 시작할 수 있는 것

2

Rule 하나 정의

기본 말투 및 보안 원칙
(AGENTS.md)



Skill 하나 구축

반복되는 특정 작업 지침
(Daily Triage 등)



3

Workflow 하나 실행

단계별 승인이 포함된
실행 가능한 Runbook

CORE PHILOSOPHY

"AI 에이전트를 잘 쓰는 법은 더 많은 프롬프트가 아니라,
더 좋은 **작업 구조(Workflow Structure)**를 만드는 것입니다."

| 향후 확장 가능성 및 미래 방향

지금 구축한 Workflow Harness를 기반으로 발전시켜 나갈 기술 로드맵

● 단기 확장 — ● 중기 확장 — ● 미래 비전



연구노트 자동화

단기

Obsidian + Cline Memory Bank 연동으로 실험 기록·문헌 요약·연구 진행 상황을 자동 정리하고 세션 간 맥락을 유지

💡 Obsidian · Memory Bank · Daily Notes



도메인별 Skillset 확장

단기

연구 분야별 특화 Skill 파일 축적 — 논문 리뷰, 코드 분석, 데이터 전처리, 보고서 작성 등 도메인 맞춤 지침 재사용

💡 SKILL.md · On-demand · 재사용



DevOps / MLOps 파이프라인 보조

중기

실험 추적, 모델 버전 관리, CI/CD 연동, GitHub PR 리뷰 보조 — Cline CLI headless mode로 자동화 파이프라인 확장

💡 CI/CD · PR Review · GitHub Actions



Materials Informatics 연구 워크플로 연결

중기

계산 연구 데이터 분석, 시뮬레이션 결과 요약, 문헌 맥락 연결 — Computational research workflow와 Cline 에이전트 통합

💡 Computational Research · Data Pipeline · Simulation



Multi-agent & Role-based Workflow

미래

역할별 에이전트 분리 (Researcher / Reviewer / DevOps) — 단순 Workflow를 먼저 안정화한 후 단계적으로 복잡도 확장

💡 Multi-agent · Role-based · Orchestration

💡 복잡한 자동화보다 **단순하고 안정적인 Workflow를 먼저 구축**하는 것이 핵심 — 잘 정리된 Rules와 Skillset이 쌓일수록 확장은 자연스럽게 이루어집니다

I Q&A 및 질의응답

강의 내용에 대한 궁금증을 해소하고 실무 적용을 위한 구체적인 논의를 진행합니다.



Any Questions?

에이전틱 워크플로 구축과
실무 자동화에 대해 자유롭게 질문해 주세요.

주요 논의 포인트 (Discussion Starters)



왜 일반 챗봇이 아니라 Cline인가?

파일/터미널/MCP 접근 권한 및 Task History 기반의 실행력 차이



Skills와 Prompt의 핵심적인 차이는?

일회성 요청이 아닌 재사용 가능한 문서형 운영 지침으로서의 역할



워크플로 운영 시 가장 먼저 망가지는 지점은?

컨텍스트 과부하 관리 및 오래된 Rule 업데이트의 중요성



현실적으로 당장 시작할 수 있는 방법은?

Rule 1개와 일일 보고서(Daily-report) 워크플로 1개부터 시작하기



온프레미스 LLM 없이도 구축이 가능한가?

Local Model 구성 가능성 및 거버넌스 설계의 필요성

I 부록 1: 전역 운영 규칙 예시 (AGENTS.md)

에이전트의 기본 행동 원칙과 보안 정책을 정의한 마크다운 샘플

i AGENTS.md의 역할

모든 워크스페이스에 공통으로 적용되는 '운영 헌법'입니다.
Cline이 어떤 언어로 말할지, 어떤 위험한 행동을 피해야 할지 전역적으로 통제합니다.

주요 설정 항목

🗨 Language & Format
기본 언어 설정 및 표준 출력 구조

🛡 Security Rules
민감 정보 보호 및 파괴적 작업 제약

💬 Interaction Style
단계별 확인 및 불확실성 처리 방식

💡 Tip: 이 파일을 Cline의 Global Rules 설정 경로에 저장하여 모든 프로젝트에 자동 적용하십시오.

AGENTS.md — Markdown

AGENTS.md — Working Agreements

Language & Format

- 기본 응답 언어: 한국어 (기술 용어는 영어 병기 허용)
- 결과물 구조: 요약 → 액션 아이템 → 다음 단계

Security Rules

- 메일 전송, Git push, 배포: 반드시 명시적 승인 후 제안만 함
- 실제 credential, endpoint, 개인정보는 절대 출력하지 않음
- 민감 정보 발견 시 익명화 샘플로 대체

Interaction Style

- 단계별 진행, 각 단계마다 확인 요청
- 불확실한 경우 추정 대신 질문

이 규칙은 시스템의 기본값이 되어야 하며,
모든 하위 프로젝트 규칙(.clinerules)에 우선하여 고려됩니다.

부록 2: 프로젝트별 로컬 규칙 예시 Appendix

특정 연구 프로젝트의 코딩 컨벤션, Git 워크플로, 보고서 형식을 정의한 프로젝트 맞춤형 Local Rules 사례

 .clinerules/10-research-project-policy.md

적용 범위 (Local Priority)

해당 Workspace에서만 활성화 · Global Rules를 덮어씀

Coding Convention

- ▶ Python: PEP 8 + type hints 필수
- ▶ Fortran: 모듈 단위 분리, `implicit none` 명시
- ▶ Docstring: Google style

Git Workflow

- ▶ Commit: `feat/fix/docs/refactor/test`
- ▶ Branch: `feature/[issue-number]-[desc]`
- ▶ Merge Request: 코드 리뷰 1인 이상 필수

Report Format

- ▶ 섹션 구조: 목적 → 방법 → 결과 → 다음 단계
- ▶ 수치 데이터: 단위 명시 필수

 10-research-project-policy.md  Local Scope Only

.clinerules/10-research-project-policy.md

Project: [연구 프로젝트명]

Coding Convention

- Python: PEP 8, type hints 필수
- Fortran: 모듈 단위 분리, `implicit none` 명시
- 함수 docstring: Google style

Git Workflow

- Commit message: `feat/fix/docs/refactor/test` 접두사 사용
- Branch naming: `feature/[issue-number]-[short-desc]`
- Merge request: 코드 리뷰 1인 이상 필수

 같은 Cline이 연구 프로젝트와 DevOps 프로젝트에서 다르게 행동하도록 설계 · Local rules로 프로젝트 특성에 맞는 절차 정의 가능

부록 3: 일일 보고서 마크다운 템플릿

📄 Daily Report Template

에이전트가 일관된 형식으로 보고서를 작성할 수 있도록 돕는 구조화된 템플릿 예시입니다.

daily-report-{{DATE}}.md

```
# Daily Report - {{DATE}}

## 오늘 한 일
- [ ] 항목 1 (근거: git commit / note 링크)
- [ ] 항목 2

## 이슈 및 Blocker
- 문제: [설명]
- 영향: [범위]
- 대응: [조치 또는 질문]

## 내일 할 일
- [ ] 우선순위 1
- [ ] 우선순위 2

## 도움이 필요한 항목
- [항목]: [담당자 또는 필요한 것]
```

📌 Skill: `daily-report` · 저장 경로: `notes/{{DATE}}-report.md`

📖 섹션별 역할 안내

✅ 오늘 한 일 증거 기반

완료 항목마다 **git commit** 또는 **note 링크**를 근거로 첨부합니다. 에이전트가 git log와 notes를 참조해 자동 수집합니다.

⚠️ 이슈 및 Blocker 3단 구조

문제 → 영향 범위 → 대응 조치의 3단 구조로 이슈를 서술합니다. 모호한 상태가 아닌 명확한 액션으로 정리합니다.

➡️ 내일 할 일 우선순위 정렬

다음 날 태스크를 우선순위 순서로 나열합니다. 이 항목은 **다음 세션의 activeContext**로 자동 전환됩니다.

🛠️ 도움이 필요한 항목 협업 신호

담당자 또는 필요한 리소스를 명시합니다. Human-in-the-loop의 **협업 요청 지점**으로 활용됩니다.

💡 **Skill 연동:** `daily-report` Skill이 notes, git log, Memory Bank를 자동 수집하여 이 템플릿을 채웁니다.

부록 5: 에이전트 상호작용 경계 및 금지 규칙

Appendix 5

자동 실행 허용 범위와 사람의 승인이 필요한 작업을 명시한 보안 경계 규칙 예시 — `.clinerules/20-boundaries.md`

`.clinerules/20-boundaries.md` | Interaction Boundary Rule

허용된 자동 실행 Auto-Allowed Actions

사용자 승인 없이 실행 가능

- ✓ 파일 읽기 및 텍스트 요약
- ✓ 분류, 초안 작성, 형식 변환
- ✓ `git status`, `git log`, `git diff` 실행
- ✓ 로컬 읽기 전용 테스트 실행
- ✓ 메모, 노트 내용 검색 및 인덱싱

원칙

읽기 전용 + 부작용 없는 작업만 자동 실행 허용

반드시 승인 필요 Requires Human Approval

실행 전 명시적 확인 요청

- ⚠ 파일 쓰기, 삭제, 이름 변경
- ⚠ `git add`, `git commit`, `git push`
- ⚠ 이메일 전송, 외부 API 호출
- ⚠ 배포(Deploy), 서버 재시작
- ⚠ 패키지 설치, 환경 변수 수정

원칙

상태 변경 · 외부 영향 · 되돌리기 어려운 작업은 반드시 사람이 승인

절대 금지 Strictly Prohibited

어떤 경우에도 실행 불가

- ✗ Credential, API Key, Password 출력 또는 로깅
- ✗ 개인정보·미공개 연구 데이터 외부 전송
- ✗ 승인 없는 프로덕션 배포 또는 데이터 삭제
- ✗ 내부 네트워크 정보, Endpoint URL 노출
- ✗ 규칙 파일 자체 수정 또는 우회 시도

원칙

보안·법적 위험 또는 되돌릴 수 없는 피해 가능성이 있는 모든 행동

“ 자동화(Automation) ≠ 자율성(Autonomy) — 경계 설계가 신뢰의 기반입니다 ”

■ 자동 허용 ■ 승인 필요 ■ 절대 금지